Introducción a SystemVerilog

Arquitectura de Computadoras 2026

De HDL a compuertas

Simulación

- Se aplican entradas al circuito.
- Se verifican las salidas para comprobar su corrección.
- Se ahorran millones de dólares depurando en simulación en lugar de en hardware.

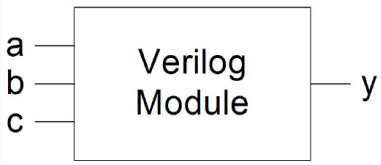
Síntesis

- Transforma el código HDL en una *netlist* que describe el hardware (es decir, una lista de compuertas y los cables que las conectan).
- El sintetizador lógico puede realizar optimizaciones para reducir la cantidad de hardware requerido.

Módulos en SystemVerilog

Dos tipos de módulos:

- Comportamental (*behavioral*): describe qué hace un módulo.
- Estructural (*structural*): describe cómo está construido a partir de módulos más simples.



SystemVerilog Comportamental

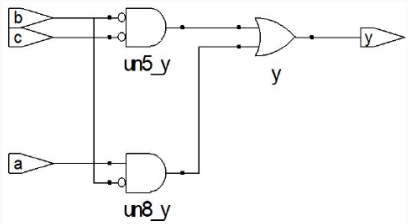
Modelado comportamental: describir un módulo en términos de las relaciones entre entradas y salidas.

- `module`/`endmodule`: requeridos para comenzar/terminar un módulo.
- El módulo comienza con el nombre (`example`) y un listado de las entradas/salidas.
- Las señales `logic`, como las entradas y salidas, son variables booleanas (0 ó 1). También pueden tener valores flotantes (`z`) e indefinidos (`x`).
- La sentencia `assign` describe lógica combinacional.

```
// Boolean function y = a'b'c'+ ab'c'+ ab'c
module example(input  logic a, b, c,
               output logic y);
  assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;
endmodule
```

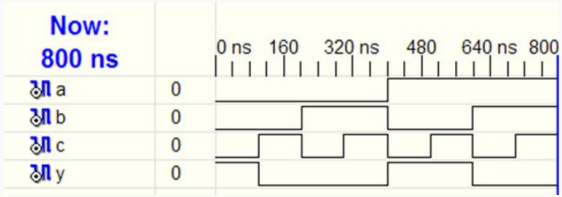
Síntesis de HDL

```
// Boolean function y = a'b'c'+ ab'c'+ ab'c
module example(input  logic a, b, c,
               output logic y);
  assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b &  c;
endmodule
```



Simulación de HDL

```
// Boolean function y = a'b'c'+ ab'c'+ ab'c
module example(input  logic a, b, c,
               output logic y);
  assign y = ~a&~b&~c | a&~b&~c | a&~b&c;
endmodule
```



a	b	c	y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Sintaxis de SystemVerilog

- **Distingue mayúsculas de minúsculas**
  Ejemplo: `reset` y `Reset` no son la misma señal.
- **Los nombres no pueden empezar con números**
  Ejemplo: `2mux` es un nombre inválido.
- **Los espacios en blanco se ignoran**
- **Comentarios:**

```
// comentario de una linea
/* comentario
   multilinea */
```

Operadores de SystemVerilog y precedencia

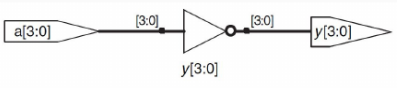
Op	Significado
`~`	NOT
`*, /, %`	MUL, DIV, MOD
`+, -`	PLUS, MINUS
`«, »`	Desplazamiento lógico izq./der.
`«<, »>`	Desplazamiento aritmético izq./der.
`<, <=, >, >=`	Comparación relativa
`==, !=`	Comparación de igualdad
`&`	AND
`^`	XOR
`|`	OR
`?:`	Condicional

(orden de mayor a menor precedencia)

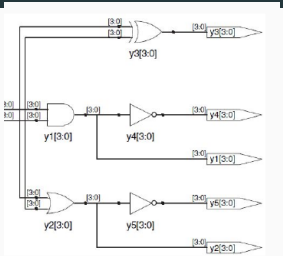
Operadores bit a bit

- Los operadores bit a bit actúan sobre señales de un bit o sobre buses multi-bit.
- `a[3:0]` representa un bus de 4 bits. Los bits, del más significativo al menos significativo, son `a[3]`, `a[2]`, `a[1]` y `a[0]`. El bit menos significativo tiene el número de bit más chico.

```
module inv  (input logic [3:0] a,
             output logic [3:0] y);
    assign y = ~a;
endmodule
```



Operadores bit a bit



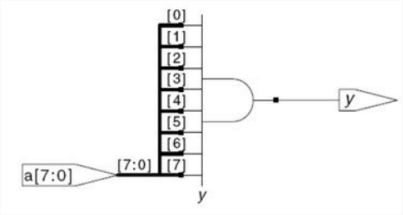

```
module gates(input  logic [3:0] a, b,
             output logic [3:0] y1, y2,
                                y3, y4, y5);
  /* Five different two-input logic
     gates acting on 4 bit busses */
  assign y1 = a & b;  // AND
  assign y2 = a | b;  // OR
  assign y3 = a ^ b;  // XOR
  assign y4 = ~(a & b); // NAND
  assign y5 = ~(a | b); // NOR
endmodule
```

Cada vez que cambian las entradas del lado derecho del = en una sentencia de asignación continua, se recalcula la salida del lado

Operadores de reducción

- Los operadores de reducción implican una compuerta de múltiples entradas actuando sobre un único bus.

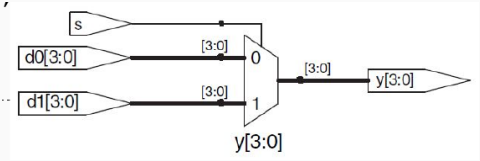
```
module and8 (input  logic [7:0] a,
             output logic  y);
    assign y = &a;
// &a es mucho mas facil de escribir que
// assign y = a[7]&a[6]&a[5]&a[4]&
//            a[3]&a[2]&a[1]&a[0];
endmodule
```



Asignación condicional

- Las asignaciones condicionales seleccionan la salida entre varias alternativas en base a una entrada llamada condición.
- El operador condicional ?: se llama operador ternario, porque toma tres entradas.

```
module mux2(input  logic [3:0] d0, d1,
            input  logic       s,
            output logic [3:0] y);
  assign y = s ? d1 : d0;
endmodule
```



Operadores condicionales anidados

```
// 4:1 multiplexer
module mux4 (input logic [3:0] d0, d1, d2, d3,
             input logic [1:0] s,
             output logic [3:0] y);

    assign y = s[1] ? (s[0] ? d3 : d2):(s[0] ? d1 : d0);
endmodule
```

/* Si `s[1]` es 1, el multiplexor elige la primera expresión, `(s[0] ? d3 : d2)`. Esta expresión a su vez elige entre `d3` o `d2` según `s[0]` (`y = d3` si `s[0]` es 1 y `d2` si `s[0]` es 0). Si `s[1]` es 0, el multiplexor elige similarmente la segunda expresión, que da `d1` o `d0` según `s[0]`. */

Números

Formato: `N'Bvalue`

- **N** = cantidad de bits, **B** = base.
- SystemVerilog soporta `'b` para binario, `'o` para octal, `'d` para decimal y `'h` para hexadecimal.
- `N'B` es opcional pero recomendado (por defecto es decimal).
- `'0` y `'1`: llenan un bus con todos 0s o todos 1s, respectivamente.

- `z`: indica un valor flotante.
- `x`: indica un nivel lógico inválido.

Números - Ejemplos

Números	Bits	Base	Val	Almacenado
3'b101	3	2	5	101
'b11	?	2	3	000 ... 0011
8'b11	8	2	3	00000011
8'b1010_1011	8	2	171	10101011
3'd6	3	10	6	110
6'o42	6	8	34	100010
8'hAB	8	16	171	10101011
42	?	10	42	00 ... 0101010

Manipulación de bits - Ejemplo

```
    assign y = {a[2:1], {3{b[0]}}, a[0], 6'b100_010};
// si y es una señal de 12 bits, la sentencia anterior produce:
    y = a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0

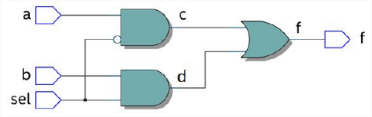
// los guiones bajos (_) se usan solo para formatear y
// facilitar la lectura. SystemVerilog los ignora.
```

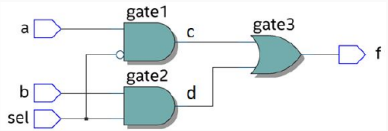
- El operador {} se usa para concatenar buses.
- {3{b[0]}} indica tres copias de b[0].
- Si y fuera más ancho que 12 bits, se colocarían ceros en los bits más significativos.

Módulos SV - tipo Comportamental vs Estructural

```
// exampleB
module exampleB
    (input  logic a, b, sel,
     output logic f);
    logic c, d;
    assign c = a & (~sel);
    assign d = b & sel;
    assign f = c | d;
endmodule
```

```
// exampleS
module exampleS(input  logic a, b, sel,
                output logic f);
    logic c, d, not_sel;
    not gate0(not_sel, sel);
    and gate1(c, a, not_sel);
    and gate2(d, b, sel);
    or gate3(f, c, d);
endmodule
```

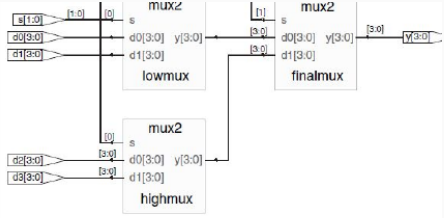




Modelado Estructural - Jerarquía y variables internas

Describir un módulo en términos de cómo está compuesto por módulos más simples.

```
module mux2(input  logic [3:0] d20,
     d21,
            input  logic s2,
            output logic [3:0] y2);
  assign y2 = s2 ? d21 : d20;
endmodule
```

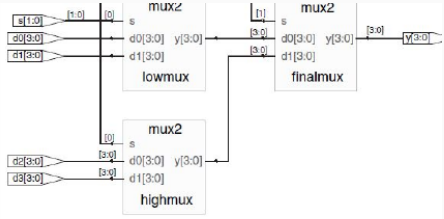



```
/* Assemble a 4:1 multiplexer from
   three 2:1 multiplexers */

module mux4(input logic [3:0] d0, d1, d2, d3,
            input logic [1:0] s,
            output logic [3:0] y);
   logic [3:0] low, high;  // internal variables
   mux2 lowmux(d0, d1, s[0], low);    // instance
     of mux2
   mux2 highmux(d2, d3, s[0], high);  // instance
     of mux2
   mux2 finalmux(low, high, s[1], y);// instance
     of mux2
endmodule
```

Modelado Estructural - Jerarquía y variables internas

```
module mux2(input  logic [3:0] d20,
     d21,
            input  logic s2,
            output logic [3:0] y2);
  assign y2 = s2 ? d21 : d20;
endmodule
```



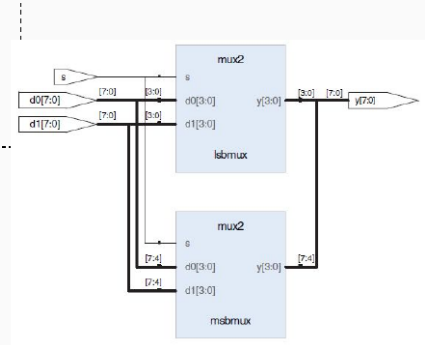

```
module mux4(input logic [3:0] d0, d1, d2, d3,
            input logic [1:0] s,
            output logic [3:0] y);
   logic [3:0] low, high;  // internal variables
   mux2 lowmux(.d20(d0), .d21(d1), .s2(s[0]), .y2(
     low));
   mux2 highmux(.s2(s[0]), .y2(high), .d20(d2), .
     d21(d3));
   mux2 finalmux(.y2(y), .d21(high), .d20(low), .
     s2(s[1]));
endmodule
```

Accediendo a partes de buses

```
/* Assemble an 8-bit wide 2:1 multiplexer
using two 4-bit 2:1 multiplexers */

module mux2_8(input logic [7:0] d0, d1,
              input logic s,
              output logic [7:0] y);

    mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
    mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);
endmodule
```



Bloque procedural *always*

En SystemVerilog, en las sentencias `always` las señales mantienen su valor anterior hasta que ocurre un evento en la lista de sensibilidad que causa explícitamente su cambio.

Estructura general:

```
always @(sensitivity list)
    statement;
```

Cada vez que ocurre el evento en `sensitivity list`, se ejecuta `statement`

always - Lógica Combinacional

Para representar **lógica combinacional** con un bloque procedural `always` de propósito general:

- La palabra clave `always` debe ir seguida de un control de evento (el símbolo `@`).
- La lista de sensibilidad del control de evento no puede contener calificadores `posedge` o `negedge`.
- La lista de sensibilidad debe incluir todas las entradas del bloque procedural.
- El bloque procedural no puede contener otros controles de evento.
- Todas las variables escritas por el bloque procedural deben actualizarse para todas las condiciones de entrada posibles.
- Ninguna variable escrita por el bloque procedural puede ser escrita por otro bloque procedural.

Bloque procedural *always* especializado

always_comb, **always_latch** y **always_ff**
indican la intención de diseño.

- Con **always_comb** no es necesario especificar una lista de sensibilidad, porque las herramientas de software saben que la intención es representar lógica combinacional. Esta lista de sensibilidad inferida incluye toda señal leída por el bloque procedural.
- Un bloque procedural **always_comb** se dispara automáticamente una vez en el instante de simulación cero.
- Si el contenido de un bloque procedural especializado no cumple las reglas de ese tipo de lógica, las herramientas de software pueden emitir mensajes de advertencia.

Lógica Combinacional - *always_comb*

```
module gates(input  logic [3:0] a, b,
             output logic [3:0] y1, y2, y3, y4, y5);
   always_comb      // need begin/end because there is
      begin          // more than one statement in always
      y1 = a & b;  // AND
      y2 = a | b;  // OR
      y3 = a ^ b;  // XOR
      y4 = ~(a & b); // NAND
      y5 = ~(a | b); // NOR
      end
endmodule
```

Problema con *always_comb*

Ejemplo en SystemVerilog:

```
always_comb
    if (en) y = a;
```

- Las herramientas de software pueden emitir una advertencia indicando que se requeriría un *latch* para realizar la funcionalidad del bloque procedural en hardware.
- La forma correcta de modelar el ejemplo como lógica combinacional sería incluir una rama **else** para que la salida 'y' se actualice para **todas las condiciones** de 'en'.

always - Lógica Secuencial (con Flip-Flops)

Para representar **lógica secuencial** con un bloque procedural `always` de propósito general:

- La palabra clave `always` debe ir seguida de un control de evento sensible a flanco (el símbolo `@`).
- Todas las señales en la lista de sensibilidad del control de evento deben estar calificadas con `posedge` o `negedge`.
- El bloque procedural no puede contener otros controles de evento.
- Ninguna variable escrita por el bloque procedural puede ser escrita por otro bloque procedural.

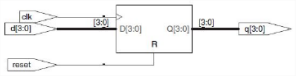
Lógica Secuencial - *always_ff*

```
always_ff @(posedge clock, negedge resetN)
    if (!resetN) q <= 0;
    else q <= d;
```

- El bloque procedural **always_ff** requiere que cada señal en la lista de sensibilidad esté calificada con **posedge** o **negedge**. Esto es un requisito de síntesis para la lista de sensibilidad de lógica secuencial.
- Que esta regla sea un requisito sintáctico ayuda a asegurar que los resultados de simulación coincidan con los de síntesis.

Lógica Secuencial - Registro con reset

Reset asíncrono:



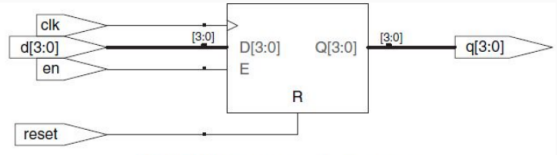

```
module flopr(input logic clk,
             input logic reset,
             input logic [3:0] d,
             output logic [3:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule
```

Reset síncrono:

```
module floprsync(input logic clk,
                 input logic reset,
                 input logic [3:0] d,
                 output logic [3:0] q);
    always_ff @(posedge clk)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule
```

Lógica Secuencial - Registro con reset y enable

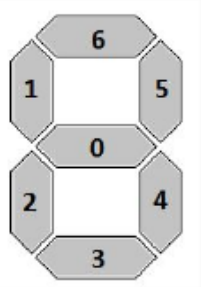
```
module flopenr(input logic clk,
               input logic reset,
               input logic en,
               input logic [3:0] d,
               output logic [3:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else if (en) q <= d;
endmodule
```



Sentencia *case* - Lógica Combinacional

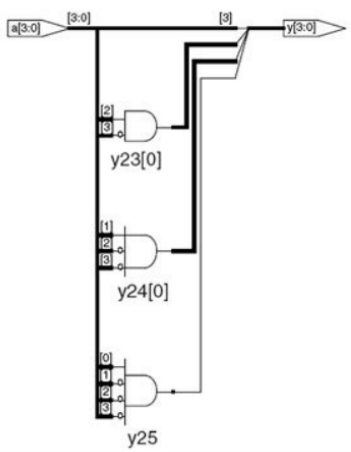
```
module sevenseg(input  logic [3:0] data,
                output logic [6:0] segments);
  always_comb
    case (data)
     //                    654_3210
     0: segments =      7'b111_1110;
     1: segments =      7'b011_0000;
     2: segments =      7'b110_1101;
     3: segments =      7'b111_1001;
     4: segments =      7'b011_0011;
     5: segments =      7'b101_1011;
     6: segments =      7'b101_1111;
     7: segments =      7'b111_0000;
     8: segments =      7'b111_1111;
     9: segments =      7'b111_0011;
     default: segments = 7'b000_0000; // required
    endcase
endmodule
```

La sentencia case implica lógica combinacional solo si se describen todas las combinaciones de entrada posibles



Sentencia *casez* - Lógica Combinacional

```
module priority_casez(input  logic [3:0] a,
                      output logic [3:0] y);
    always_comb
    casez(a)
      4'b1???: y = 4'b1000; // ?=don't care
      4'b01??: y = 4'b0100;
      4'b001?: y = 4'b0010;
      4'b0001: y = 4'b0001;
      default: y = 4'b0000;
    endcase
endmodule
```



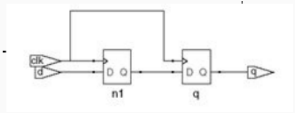
Asignaciones dentro de bloques procedurales *always*

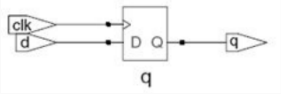
<= es asignación **no bloqueante**: ocurre simultáneamente con las demás (para **lógica secuencial**)

= es asignación **bloqueante**: ocurre en el orden en que aparece en el archivo (para **lógica combinacional**)

```
// Good synchronizer using
// nonblocking assignments
module syncgood(input  logic clk,
                input  logic d,
                output logic q);
  logic n1;
  always_ff @(posedge clk)
    begin
       n1 <= d;  // nonblocking
       q  <= n1; // nonblocking
    end
endmodule
```

```
// Bad synchronizer using
// blocking assignments
module syncbad(input logic clk,
               input  logic d,
               output logic q);
  logic n1;
  always_ff @(posedge clk)
    begin
       n1 = d;  // blocking
       q  = n1; // blocking
    end
endmodule
```





Módulos parametrizados

2:1 mux:

```
module mux2
  #(parameter width = 8)  // nombre y valor por defecto
  (input  logic [width-1:0] d0, d1,
   input  logic s,
   output logic [width-1:0] y);
   assign y = s ? d1 : d0;
endmodule
```

Instancia con bus de 8 bits (usa el valor por defecto):

```
mux2 myMux(d0, d1, s, out);
```

Instancia con bus de 12 bits:

```
mux2 #(12) lowmux(d0, d1, s, out);
```

Arrays desempaquetados

La sintaxis básica de una declaración de array desempaquetado es:

```
<data_type> <element_size> <array_name> <array_dimensions>
```

Por ejemplo:

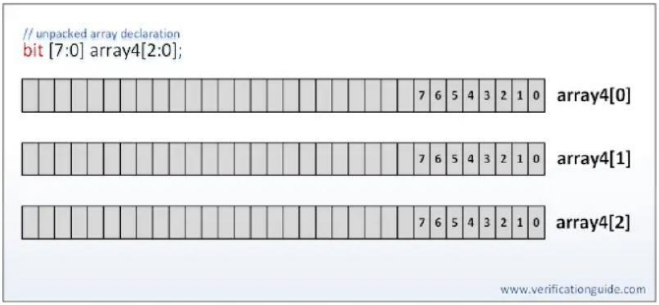
```
logic [7:0] table [3:0]; // array "table"
                         // tiene 4 elementos de 8 bits
```

- Con arrays desempaquetados, cada elemento del array puede almacenarse independientemente de los demás.
- Se usan arrays desempaquetados para modelar arrays donde típicamente se accede a un elemento a la vez, como en RAMs y ROMs.

Ejemplo de array desempaquetado

```
bit [7:0] array4[2:0];
```

Below diagram shows storing unpacked array as a **non-contiguous** set of bits.



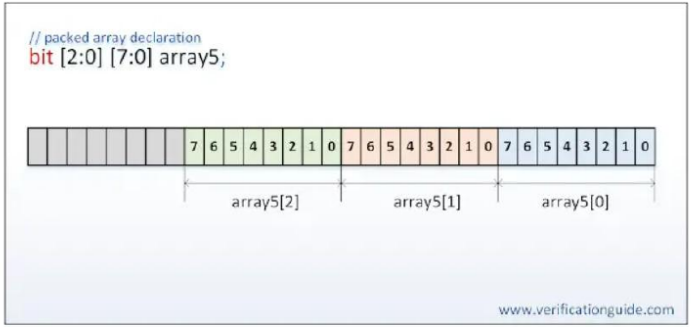
SystemVerilog Unpacked array

Fuente: verificationguide.com

Ejemplo de array empaquetado

```
bit [2:0] [7:0] array5;
```

The below diagram shows storing packed array as a **contiguous** set of bits.



SystemVerilog Packed Array

Inicialización de arrays desempaquetados

- Los arrays desempaquetados pueden inicializarse en su declaración, usando una lista de valores encerrada entre llaves '{ y } para cada dimensión del array.
- La asignación requiere conjuntos anidados de llaves que coincidan exactamente con las dimensiones del array.

```
logic [31:0] ROM [0:5] = '{32'h8b020021,
                           32'h8b000000,
                           32'h8b000000,
                           32'hf8008001,
                           32'h91000461,
                           32'h8b000000};
```

Ejemplos de inicialización de arrays desempaquetados

```
                logic [0:7] a1 [0:1023] = '{default:8'h55};
/* Specifying a default value to initialize all the elements,
or a slice, of an unpacked array*/
```

```
logic [3:0] a [0:3];
  a = '{'{0,1,0,1}, '{1,0,0,0}, '{1,1,1,1}, '{0,0,1,0}};
                        // assign a list of values to the full array
  a[3] = 4'h5;       // assign list of values to slice of the array
  a[1][0] = 1'b1;    // assign to one element
```

```
logic a1 [7:0][1023:0];    // unpacked array
logic a2 [8:1][1024:1];    // unpacked array
  a2 = a1;           // copy an entire array
  a2[3] = a1[0];     // copy a slice of an array
```

Testbenches

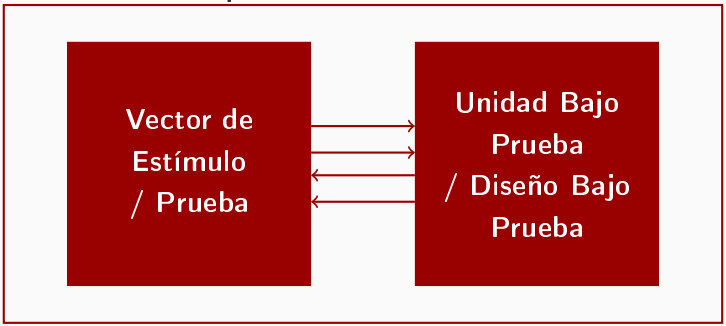
Los módulos sintetizables describen el hardware. El testbench contiene código para aplicar entradas a un módulo, verificar si los resultados de salida son correctos, e imprimir las discrepancias entre las salidas esperadas y las reales.

- Módulo HDL que prueba otro módulo, el dispositivo bajo prueba (*dut*)
- No sintetizable
- Tipos:
  - → Simple
  - → Auto-verificable (*self-checking*)
  - → Auto-verificable con vectores de prueba

Testbenches

- El testbench contiene sentencias para aplicar entradas al **DUT** y, idealmente, para verificar que se produzcan las salidas correctas.
- Los patrones de entrada y salida deseada se llaman **vectores de prueba**.

Arquitectura de Testbench



Testbench - Ejemplo

Escribir código SystemVerilog para implementar la siguiente función en hardware:

$$y = b'c' + ab'$$

```
module sillyfunction(input  logic a, b, c,
                     output logic y);
  assign y = ~b & ~c | a & ~b;
endmodule
```

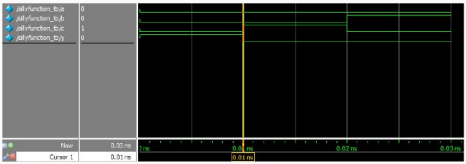
Testbench simple

```
module testbench1();
  logic a, b, c; // internal variables
  logic y;
  // instantiate device under test:
  sillyfunction dut(a, b, c, y);
  // apply inputs one at a time:
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
    c = 1; #10;
    a = 1; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
    c = 1; #10;
  end
endmodule
```

Testbench - Retardos

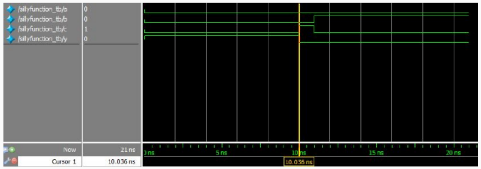
- El símbolo # se usa para indicar la cantidad de unidades de retardo.
- Unidad de escala de tiempo por defecto en Quartus Prime = 1ps.

```
module sillyfunction_tb();
  logic  a, b, c;
  logic  y;
 // instantiate device under test
  sillyfunction dut(a, b, c, y);
 // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
  end
endmodule
```



Testbench - Retardos

```
module sillyfunction_tb();
  logic  a, b, c;
  logic  y;
 // instantiate device under test
  sillyfunction dut(a, b, c, y);
 // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10 ns;
    c = 1; #1000;
    b = 1; c = 0; #10 ns;
  end
endmodule
```

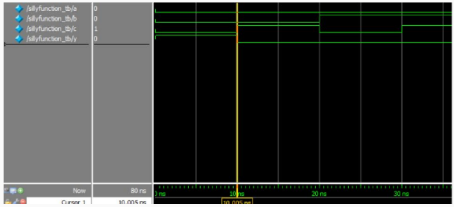


Testbench - Retardos

`'timescale 1ns / 10ps`: se le indica a la herramienta de software que use unidades de tiempo de 1 nanosegundo, y una precisión de 10 picosegundos

```
'timescale 1ns / 10ps

module sillyfunction_tb();
  logic  a, b, c;
  logic  y;
 // instantiate device under test
  sillyfunction dut(a, b, c, y);
 // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
  end
endmodule
```



Testbench auto-verificable

```
module testbench2();
  logic  a, b, c;
  logic y;
  // instantiate dut
  sillyfunction dut(a, b, c, y);

  /* apply inputs and check results
     one at a time */
  initial begin
    a = 0; b = 0; c = 0; #10;
    if (y !== 1) $display("000 failed.");
    c = 1; #10;
    if (y !== 0) $display("001 failed.");
    b = 1; c = 0; #10;
    if (y !== 0) $display("010 failed.");
    c = 1; #10;
    ...
```

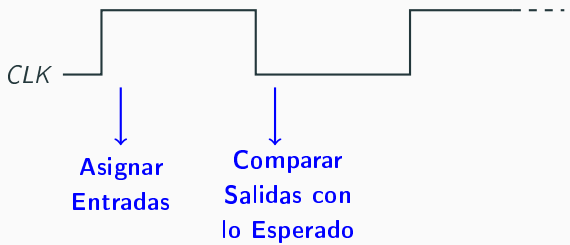
```
...
if (y !== 0) $display("011 failed.");
a = 1; b = 0; c = 0; #10;
if (y !== 1) $display("100 failed.");
c = 1; #10;
if (y !== 1) $display("101 failed.");
b = 1; c = 0; #10;
if (y !== 0) $display("110 failed.");
c = 1; #10;
```

Testbench con vectores de prueba (TwT)

Reloj del testbench:

→ asignar entradas (en flanco de subida)

→ comparar salidas con las salidas esperadas (en flanco de bajada)



El reloj del testbench también se usa como reloj para los circuitos secuenciales síncronos.

TwT 1. Leer vectores de prueba en un Array

```
module testbench4();
  logic     clk, reset;
  logic     a, b, c, yexpected;
  logic     y;
  logic [31:0] vectornum, errors; // bookkeeping variables
  logic [3:0] testvectors [7:0] = '{ 4'b000_1,// array de vectores de prueba
                                     4'b001_0,
                                     4'b010_0,
                                     4'b011_0,
                                     4'b100_1,
                                     4'b101_1,
                                     4'b110_0,
                                     4'b111_0};
...
```

TwT 2. Generar el reloj

```
  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // generate clock
  always     // no sensitivity list, so it always executes
    begin
        clk = 1; #5; clk = 0; #5;
    end

  initial      // at start of test pulse reset
    begin
        vectornum = 0; errors = 0; reset = 1; #27;
        reset = 0;
    end
...
```

TwT 3. Asignar entradas y salidas esperadas

```
  // apply test vectors on rising edge of clk
  always @(posedge clk)
    begin
        #1; {a, b, c, yexpected} = testvectors[vectornum];
    end
...
```

TwT 4. Comparar con las salidas esperadas

```
  // check results on falling edge of clk
  always @(negedge clk)
    if (~reset) begin // skip during reset
      if (y !== yexpected)
        begin
        $display("Error: inputs = %b", {a, b, c});
        $display("outputs = %b (%b expected)",y,yexpected);
        errors = errors + 1;
        end
    end
...

// Note: to print in hexadecimal, use %h. For example,
//       $display("Error: inputs = %h", {a, b, c});
```

TwT 4. Comparar con las salidas esperadas

```
  // increment array index and read next testvector
        vectornum = vectornum + 1;
        if (testvectors[vectornum] === 4'bx)
            begin
            $display("%d tests completed with %d errors",
                      vectornum, errors);
            $finish; // Usar $stop para que no se cierre ModelSim
            end
    end
endmodule

// === and !== can compare values that are 1, 0, x, or z.
```

Bibliografía

- S. Harris and D. Harris, “Digital Design and Computer Architecture - ARM Edition”. Elsevier, 2016.
- S. Sutherland, S. Davidmann, P. Flake and P. Moorby, “SystemVerilog for Design - A Guide to Using SystemVerilog for Hardware Design and Modeling”, Second Edition. Springer, 2006.